

Status Update & Task Conflicts

2m

- Task progress review initiated for period since Wednesday.
- User Story 14 conflict identified between Sam and Thendo.
 - Thendo intended to start Story 14 but discovered Sam already completed it.
 - Thendo required to select an alternative user story for the current sprint.
- Meeting minutes noted as being automatically recorded.
- **Agreement:** Review of individual progress to proceed sequentially across the team.

User Story Assignment & Workflow Optimization

- Sam completed User Story 14; Thendo to identify new task.
- Coordination conflict identified regarding overlapping work on the same User Stories.
- **Agreement:** Shift to a "section-based" ownership model rather than random selection to ensure feature consistency and simplify merge processes.
- Kiran to own all analytics-related User Stories for the Admin side.
- Thendo to manage integration-heavy User Stories for consistent implementation.

Sprint Progress Updates

4m

- Concerns raised regarding workload balance and tracking progress across current sprints.
- Germaine completed onboarding User Stories, specifically focusing on Admin capabilities for user creation.
- Logical grouping of tasks proposed to prevent hand-off friction and mid-feature technical debt.

Worker Onboarding & Verification

- Germaine completed three user stories regarding municipal worker roles
 - Default status for all new sign-ins is "resident"
 - Admin enters worker email via dashboard to trigger verification link

- Municipal workers verified via unique email link and redirected to worker page
- Admin functionality enabled to revoke municipal worker roles

Ward Mapping & Infrastructure

- Abdul completed the Ward Map feature (pending merge)
 - Interactive map displays all wards
 - Selection of specific wards retrieves relevant data and details
 - **Agreement:** Abdul to send pull request to main branch for final integration

Admin Analytics & Testing

7m

- Category Report page developed by Kiran using live database data
 - Metrics tracked: total requests, pending requests, resolved requests
 - Performance data: average resolution time and identification of most inefficient categories
 - Data visualized via two graph formats
- Navigation and Architecture updates
 - Routing fixes implemented across the application
 - Drop-down menu added to navigation bar for administrative analytics
- Test suite expanded with new cases located in the /tests folder

Frontend Refactoring and Testing

- Navigation and domain updates completed by Kiran Soodyall.
 - Dropdown menu added to Navigation bars for Admin analytics section.
- Test suite expanded and centralized.
 - Seven test files authored/organized within the `Tests` folder.
 - Integration of external test files (Sam's) into the primary directory.
- Component architecture reorganized.
 - Global `components` folder established to house assets previously nested within `Pages`.

Backend Integration and Geolocation

10m

- Claim request component developed for Firestore integration.
 - Allows UI-driven logging of user claims directly to the database.
- Data integration for location services.
 - External server pinging implemented to bypass JavaScript Geolocation API issues.
 - Integration of municipal, ward, and provincial data for future display capabilities.
- Backend listening endpoints established.
 - Added specific server-side endpoints to the primary file hosting the webpage.

Backend Refactoring and Image Storage

- Backend server consolidated into a single file serving both API endpoints and the frontend.
- Cloudinary identified as the long-term solution for image hosting.
- Temporary fix implemented using Data URIs to bypass backend storage requirements.
 - Enables full functionality for the "Submit Request" page and form submissions.

Sprint Requirements and Testing Assignments

12m

- Project status requires 4 total meetings and 6 user stories for completion.
- **Agreement:** Kiran to perform test coverage for all existing code currently merged into the main branch.
- Sam to migrate existing tests for the Request page into the Jest framework.
 - Specific focus on `pages/requests` folder and `request.test.js`.
 - Component-level test cases for specific commits remain pending.

Test Coverage and AI Integration

- NPM command `npm run test` identified as the method to verify test coverage and functionality for Germaine, Siby, and Thendo.
- Kiran completed test cases for all category-related components (Category Option, Category Select).
- Kiran proposed a workflow using Gemini AI to generate test suites for components and pages.
 - AI handles logic generation; manual intervention required only for fixing imports and file reference URLs.

- **Agreement:** Kiran to handle component testing while other team members focus on page-specific tests.

Backend Architecture and Firebase

15m

- Concern raised regarding front-end vulnerabilities when querying Firebase directly.
- Structural issue noted: Firebase backend code is often being mixed with front-end logic rather than isolated.
- Backend folder exists in the project structure, but usage is currently inconsistent among the team.
- **Agreement:** Backend logic should be separated from the front end to improve security and maintainability.

Backend Architecture & Security Constraints

- Firebase backend code currently resides in a folder imported directly into the frontend.
- Speaker 2 proposed migrating to a separate `server.js` architecture for `API voting district` to ensure true backend isolation.
- **Agreement:** Implementation of a separate server architecture is the "right way" but will be deprioritized due to the Monday deadline and lack of strict security requirements for this phase.
- Decision made to leave backend logic within the frontend structure if time runs out over the weekend.

UML Diagrams & Software Architecture

17m

- UML diagrams and software architecture documentation are required for submission on Monday.
- Requirement stems from Sprint guidelines and material covered in Lecture 5.
- **Agreement:** Kiran to take responsibility for starting and completing the diagram documentation.

Sprint 2 Architecture Requirements

- UML Software Architecture diagrams required for Sprint 2 per project rubric.

- Minimum of 6-7 diagrams necessary based on Lecture 5 materials.
- 4+1 Architectural View Model implementation required for submission.
 - Specific views discussed: Development, Sequence, and State diagrams.
 - Core focus on database organization and page-to-page transitions (Kiran).
- Individual sprint retrospectives must document architectural decision-making roles.

Diagram Assignment & Planning

21m

- Collaboration limited to maximum of two people to maintain efficiency.
- Reference materials located in Lecture 5 notes and Sprint 2 rubric instructions.
- **Agreement:** Germaine to draft initial diagrams for team review by tomorrow.

Architectural Deliverables & Deadlines

- Sunday deadline for completion of the 4+1 Architectural View Model.
- Components required for the model based on Lecture 5 notes:
 - Logical view
 - Development view
 - Process view
 - Physical view
 - Scenario view
- Concerns raised regarding the one-week timeframe for six user stories plus full architectural documentation.
- **Agreement:** Meeting minutes will be consolidated into a single document by Germaine.
- **Agreement:** Tasks to be delegated among the team; Kiran to review Lecture 5 materials/examples.

Component Complexity & Delegation

24m

- Process and Development views identified by Kiran as the most "deep" and difficult components.
- Logical and Scenario views perceived as relatively easier to execute.

- Team strategy involves researching documentation methods over the next few days to facilitate creation.
- **Agreement:** Team to delegate specific views to individuals to avoid bottlenecking.

Architecture View Assignments

- Comparison of architectural views identifies "Process" and "Development" as the most complex.
- **Agreement:** Kiran to handle the Process view.
- **Agreement:** Germaine to handle both the Physical and Scenario views.
- Unassigned views remaining include Development and Logical (referenced as "the rest").
- Distribution of tasks managed via Discord chat.

Academic Assessment Concerns

26m

- Discussion of FLA (Formal Logic/Automata) course structure.
- Logic chapter identified as the easiest but carries the lowest weight.
- **Agreement:** Upcoming logic test confirmed to count for 5% of the final grade.

4+1 Architectural View Assignments

- Germaine assigned to Physical and Scenario views.
- Kiran assigned to Process view.
- Logical and Development views remaining for selection by Sam.
- **Agreement:** Germaine to handle Physical and Scenario views.

Logical View & Object Modeling

- Logical view requirements involve imagining system objects despite limited current class structures.
- Proposed objects for planning: Worker, Resident, and Admin.
- AI tool Claude suggested for generating Logical view diagrams.
- Use of "DB Diagram" tool identified for exporting database table views to PDF.
- **Agreement:** Sam to handle Logical view after reviewing project notes.

Academic Context & Prerequisites

28m

- Project tasks identified as a continuation of "Database Fundamentals" and "Mobile Computing" (MC) coursework.
- Database fundamentals previously covered over one full semester.
- Diagrams previously required for a joint MC project and one other specific assignment.

Software Architecture Diagrams & Assignments

- Database fundamentals and mobile computing (MC) identified as prerequisite knowledge for project diagrams.
- Dispute over whether previous exams required drawing full diagrams or simply identifying components.
- Logical view and physical view identified by Kiran as the least complex components to complete.
- Development and process views categorized as the most difficult tasks.

Task Allocation & Component Definitions

31m

- **Agreement:** Logical view assigned to Sam.
- **Agreement:** Development view assigned to Siby.
- Development view requirements clarified as high-level component diagrams rather than specific class objects.
- Component diagrams defined as abstractions implemented by one or more classes, potentially involving database objects such as Resident ID.
- Scenario and physical view tasks remained unassigned or pending final confirmation from Speaker 3 and Speaker 4.

4+1 Architectural View Assignments

- Development View: High-level component diagrams abstraction confirmed; components represent database objects (e.g., Resident ID) rather than physical objects.
- **Agreement:** View responsibilities assigned as follows:

- Logical View: Sam
- Development View: Siby
- Process View: Kiran
- Physical View: Germaine
- Scenario View: Thendo

Iteration Management & User Stories

32m

- Sam confirmed completion of multiple user stories for the current iteration.
- **Agreement:** Team to move all completed user stories to the "Done" column immediately.
- Transition strategy: Once six functional user stories are verified, development ceases to focus on documentation and presentation prep.
- Deadline pressure: Documentation must be ready for presentation scheduled for the day after tomorrow.